

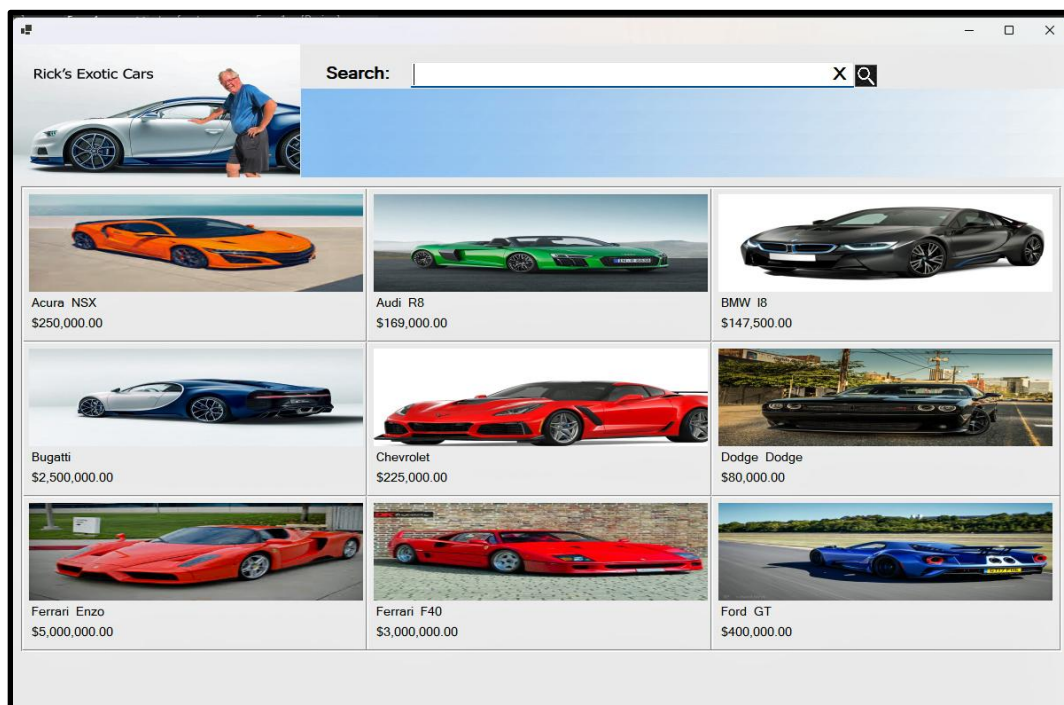
Chapter 15A: Storefront

In our next couple of examples, we are going to really start to pick up the pace. In this and the next chapter we are going to create an online storefront. Rick's Exotic Cars will present a random set of car pictures on the home page along with the car's model, manufacturer and price. If the user clicks on any of these images they are sent to a second page which presents additional information about that car including additional pictures and videos. If the user ultimately chooses to purchase a car they are taken to yet a third page. It should be great fun.

In chapters 16 A and B we are going to build some of the elements associated with an online learning and testing environment. I expect both examples are going to really challenge your skills, so let's buckle up and enjoy the ride.

We begin by creating a new Windows Forms application. I called my project storefront and this is an image of my home page. (Figure 15A.1)

15A.1: Home Page



I began by adding that logo up in the upper left-hand corner. This code should be straight forward for you by now. Not too much going on here. (Figure 15A.1) By the way, you can find all these images in a zipped resource file associated with this lesson if you are taking this course online. If you are reading this stuff from a textbook, please go to the URL listed in Chapter 1.

Figure 15A.1: AddLogo

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Size = new System.Drawing.Size(1200, 800);
    AddLogo();
}

1 reference
public void AddLogo()
{
    string my_logo = @"F:\Documents\C# The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\images\logo.jpg";
    if (File.Exists(my_logo))
    {
        Image logo = Image.FromFile(my_logo);
        PictureBox lg = new PictureBox();
        lg.Image = logo;
        lg.Size = new System.Drawing.Size(325, 150);
        lg.Location = new System.Drawing.Point(0, 0);
        lg.SizeMode = PictureBoxSizeMode.StretchImage;
        this.Controls.Add(lg);
    }
}

0 references
```

Just as an aside, you will normally not have to design these graphics yourself. I always got mine from somebody in marketing who of course had a much better eye for aesthetics. I no longer have access to a marketing team, so I put these together myself using PhotoShop. Sorry, not very good but that is the way of the world in retirement.

Next came those blue swoosh bars across the middle. Again, this code should be very easy for you by now. (Figure 15A.2)

Figure 15A.2: AddSwoosh

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Size = new System.Drawing.Size(1200, 800);
    AddLogo();
    AddSwoosh();
}

1 reference
public void AddSwoosh()
{
    string my_swoosh_file = @"F:\Documents\C# The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\images\swoosh.jpg";
    Size swoosh_size = new Size(950, 25);

    if (File.Exists(my_swoosh_file))
    {
        Image my_swoosh = Image.FromFile(my_swoosh_file);

        PictureBox pb1 = new PictureBox();
        pb1.Image = my_swoosh;
        pb1.Size = swoosh_size;
        pb1.Location = new System.Drawing.Point(326, 50);
        pb1.SizeMode = PictureBoxSizeMode.StretchImage;
        this.Controls.Add(pb1);

        PictureBox pb2 = new PictureBox();
        pb2.Image = my_swoosh;
        pb2.Size = swoosh_size;
        pb2.Location = new System.Drawing.Point(326, 75);
        pb2.SizeMode = PictureBoxSizeMode.StretchImage;
        this.Controls.Add(pb2);

        PictureBox pb3 = new PictureBox();
        pb3.Image = my_swoosh;
        pb3.Size = swoosh_size;
        pb3.Location = new System.Drawing.Point(326, 100);
        pb3.SizeMode = PictureBoxSizeMode.StretchImage;
        this.Controls.Add(pb3);

        PictureBox pb4 = new PictureBox();
        pb4.Image = my_swoosh;
        pb4.Size = swoosh_size;
        pb4.Location = new System.Drawing.Point(326, 125);
        pb4.SizeMode = PictureBoxSizeMode.StretchImage;
        this.Controls.Add(pb4);
    }
}

1 reference
```

That search bar is actually made up of four individual components. The word search is of course just a Label, the user enters their search criterion into a TextBox, the reset X is another Label and finally that magnifying glass image is in a PictureBox. My version of the code is shown here. (Figure 15A.3) Please note that I have held off on attaching the event handlers until that code is written below. The alternative would have been to write stubs.

Figure 15A.3: AddSearchBar

```
1 reference
public void AddSearchBar()
{
    string my_search_icon_file = @"F:\Documents\CS The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\images\search icon.jpg";
    Font myFont = new Font(FontFamily.GenericSansSerif, 14, FontStyle.Bold);

    Label search_label = new Label();
    search_label.Font = myFont;
    search_label.Text = "Search:";
    search_label.Location = new System.Drawing.Point(350, 20);

    Image my_search_icon = Image.FromFile(my_search_icon_file);
    PictureBox search_icon_image = new PictureBox();
    search_icon_image.Image = my_search_icon;
    search_icon_image.Location = new System.Drawing.Point(951, 23);
    search_icon_image.Size = new System.Drawing.Size(25, 25);
    search_icon_image.SizeMode = PictureBoxSizeMode.StretchImage;
    search_icon_image.BorderStyle = BorderStyle.FixedSingle;
    search_icon_image.Click += new EventHandler(DoSearch); // uncomment when we build the routine

    TextBox search_textbox = new TextBox();
    search_textbox.Name = "SearchTextbox";
    search_textbox.Font = myFont;
    search_textbox.Size = new System.Drawing.Size(500, 20);
    search_textbox.Text = "";
    search_textbox.Location = new System.Drawing.Point(450, 20);
    search_textbox.BorderStyle = BorderStyle.Fixed3D;

    Label clear_x = new Label();
    clear_x.Font = myFont;
    clear_x.Text = "X";
    clear_x.AutoSize = true;
    clear_x.Location = new System.Drawing.Point(922, 21);
    clear_x.BackColor = search_textbox.BackColor;
    clear_x.Click += new EventHandler(ClearSearch);

    this.Controls.Add(search_label);
    this.Controls.Add(clear_x);
    this.Controls.Add(search_textbox);
    this.Controls.Add(search_icon_image);
}
1 reference
```

I next went ahead and created some class definitions. Yes, these were in a separate file so I can keep things nicely organized as my complexity increases. Can you read this code and immediately envision how these classes will be used? It's time to start thinking that way. (Figure 15A.4)

Figure 15A.4: Initial Class Definitions

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace storefront
{
    2 references
    public class car_pic
    {
        public int ID;
        public string filename;
    }

    2 references
    public class car_vid
    {
        public int ID;
        public string filename;
        public string thumbnail;
    }

    0 references
    public class car
    {
        public int ID;
        public string Manufacturer;
        public string Model;
        public string description;
        public int price;
        public int horsepower;
        public double zero_to_sixty;
        public int top_speed;
        public bool passed_search;
        public int pic_count;
        public int vid_count;
        public car_pic[] pics = new car_pic[20];
        public car_vid[] vids = new car_vid[20];
    }
}
```

At this point I would have normally opened a relational database and started populating my car class with information stored in those relational tables. Well, we haven't studied relational databases yet, so I had to manually enter the information. Yuk! I bet it took me nearly an hour, and I can only imagine how many errors there are. (Figure 15A.5) Please cut and paste this stuff when you do this example.

Figure 15A.5: AddCarData

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Size = new System.Drawing.Size(1200, 800);
    AddLogo();
    AddSwosh();
    AddSearchBar();
    AddCarData();
}

1 reference
public void AddCarData()
{
    // Acura NSX
    my_cars[0] = new car();
    my_cars[0].ID = 0;
    my_cars[0].Manufacturer = "Acura";
    my_cars[0].Model = "NSX";
    my_cars[0].description = "The Acura NSX is a high-performance, two-seat sports car. The 2022 Type S variant " +
        "features a 3.0L twin-turbo V6 and a 9-speed clutch. It has a top speed of approximately " +
        "191 mph and can go from 0 to 60 mph in just over 3 seconds.";
    my_cars[0].price = 250000;
    my_cars[0].horsepower = 600;
    my_cars[0].zero_to_sixty = 3.11;
    my_cars[0].passed_search = true;
    my_cars[0].pic_count = 2;
    my_cars[0].vid_count = 0;
    my_cars[0].pics[0] = new car_pic();
    my_cars[0].pics[0].ID = 0;
    my_cars[0].pics[0].filename = @"F:\Documents\CS The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\cars\Acura NSX 1.jpg";
    my_cars[0].pics[1] = new car_pic();
    my_cars[0].pics[1].ID = 1;
    my_cars[0].pics[1].filename = @"F:\Documents\CS The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\cars\Acura NSX 2.jpg";

    my_cars[1] = new car();
    my_cars[1].ID = 1;
    my_cars[1].Manufacturer = "Audi";
    my_cars[1].Model = "R8";
    my_cars[1].description = "The Audi R8 is a high-performance, mid-engine supercar with a 17 year production run which ended " +
        "in 2024. Known for its V10 engine, all wheel drive and an available gated shifter.";
    my_cars[1].price = 160000;
    my_cars[1].horsepower = 600;
    my_cars[1].zero_to_sixty = 3.00;
    my_cars[1].passed_search = true;
    my_cars[1].pic_count = 1;
    my_cars[1].vid_count = 1;
    my_cars[1].pics[0] = new car_pic();
    my_cars[1].pics[0].ID = 0;
    my_cars[1].pics[0].filename = @"F:\Documents\CS The Whole Story\Section 3 GUI Stuff\Chapter 15A storefront\cars\Audi R8.jpg";
    my_cars[1].vid[0] = new car_vid();
}
```

I was now ready to put together that grid of car images at the bottom of the screen. My routine is shown here and was aptly named AddGrid. (Figure 15A.6) Microsoft does not recommend the nesting of TableLayoutPanel's but I did it anyway and it appears to have worked just fine. You just have to be careful as to how you reference the cell indices. Also, please note how I used the lambda operator to add an additional passed parameter to my pic.Click event handler.

Figure 15A.6 AddGrid

```
1 reference
public void AddGrid()
{
    int counter = 0;
    int start_index = random.Next(3);

    TableLayoutPanel parent_grid = new TableLayoutPanel();
    parent_grid.CellBorderStyle = TableLayoutPanelCellBorderStyle.InsetDouble;
    parent_grid.Location = new System.Drawing.Point(10, 160);
    parent_grid.Size = new System.Drawing.Size(1175, 525);
    parent_grid.ColumnCount = 3;
    parent_grid.RowCount = 3;
    this.Controls.Add(parent_grid);

    ToolTip tl = new ToolTip();

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            TableLayoutPanel child_grid = new TableLayoutPanel();
            child_grid.Size = new Size(380, 165);
            child_grid.RowCount = 3;
            child_grid.ColumnCount = 1;
            child_grid.CellBorderStyle = TableLayoutPanelCellBorderStyle.None;
            parent_grid.Controls.Add(child_grid);

            PictureBox pic = new PictureBox();
            pic.Name = "pic_" + i + "_" + j;
            pic.Size = new Size(395, 110);
            pic.SizeMode = PictureBoxSizeMode.StretchImage;

            tl.SetToolTip(pic, "Click any image to see details.");

            // add event handler and delegate via lambda operator
            if (i == 0 && j == 0) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 0].ID);
            else if (i == 0 && j == 1) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 1].ID);
            else if (i == 0 && j == 2) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 2].ID);
            else if (i == 1 && j == 0) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 3].ID);
            else if (i == 1 && j == 1) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 4].ID);
            else if (i == 1 && j == 2) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 5].ID);
            else if (i == 2 && j == 0) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 6].ID);
            else if (i == 2 && j == 1) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 7].ID);
            else if (i == 2 && j == 2) pic.Click += (sender2, e2) => ImageWasClicked(sender2, e2, my_cars[start_index + 8].ID);

            child_grid.Controls.Add(pic, 0, 0);

            // add label with text set to manufacturer + model to child grid
            Label l1 = new Label();
            l1.Name = "l1_" + i + "_" + j;
            Font my_font = new Font(FontFamily.GenericSansSerif, 10, FontStyle.Regular);
            l1.Font = my_font;
            child_grid.Controls.Add(l1, 0, 1);

            // add label with text set to price to child grid
            Label l2 = new Label();
            l2.Name = "l2_" + i + "_" + j;
            l2.Font = my_font;
            child_grid.Controls.Add(l2, 0, 2);

            counter++;
        }
        AddCarsToGrid(start_index);
    }
}
```

I next had to write the AddCarsToGrid routine. All this nesting is probably why Microsoft discourages the nesting of TableLayoutPanels. (Figure 15A.7) Took me a while to get this one working properly but I finally got there. Just take your time and keep things as simple and as straight forward as possible.

15A.7: AddCarsToGrid

```

}

// reference
private void AddCarsToGrid(int start_index)
{
    int counter = 0;

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            foreach (Control c in this.Controls)
            {
                if (c is TableLayoutPanel) // parent
                {
                    foreach (Control c2 in c.Controls)
                    {
                        if (c2 is TableLayoutPanel) // child
                        {
                            foreach (Control c3 in c2.Controls)
                            {
                                if (c3 is PictureBox)
                                {
                                    PictureBox p = (PictureBox)c3;
                                    if (p.Name == "pic_" + i + "_" + j)
                                    {
                                        Image img = Image.FromFile(my_cars[start_index + counter].pics[0].filename);
                                        p.Image = img;
                                    }
                                }

                                if (c3 is Label)
                                {
                                    Label l1 = (Label)c3;
                                    if (l1.Name == "l1_" + i + "_" + j)
                                    {
                                        l1.Text = my_cars[start_index + counter].Manufacturer;
                                    }
                                    else if (l1.Name == "l2_" + i + "_" + j)
                                    {
                                        l1.Text = my_cars[start_index + counter].price.ToString("C", new CultureInfo("en-US"));
                                        counter++;
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

9 references
```

I next had to write the event handlers associated with our search bar. I started with the ClearCars routine shown here. (Figure 15A.8) Again we see all this nesting which requires quite a bit of attention. With the ClearCars and AddCarsToGrid routines now available the ClearSearch routine associated with that “X” Label is now super straightforward. (Figure 15A.9)

15A.8: ClearCars

```
2 references
private void ClearCars()
{
    foreach (Control c in this.Controls)
    {
        if (c is TableLayoutPanel) // parent TableLayoutPanel
        {
            foreach (Control d in c.Controls)
            {
                if (d is TableLayoutPanel) // child TableLayoutPanel
                {
                    foreach (Control e in d.Controls)
                    {
                        if (e is PictureBox)
                        {
                            PictureBox pbl = (PictureBox)e;
                            pbl.Image = null;
                        }
                        else if (e is Label)
                        {
                            Label li = (Label)e;
                            li.Text = "";
                        }
                    }
                }
            }
        }
    }
}
```

Figure 15A.9: ClearSearch

```
Label clear_x = new Label();
clear_x.Font = myFont;
clear_x.Text = "X";
clear_x.AutoSize = true;
clear_x.Location = new System.Drawing.Point(922, 21);
clear_x.BackColor = search_textbox.BackColor;
clear_x.Click += new EventHandler(ClearSearch);

this.Controls.Add(search_label);
this.Controls.Add(clear_x);
this.Controls.Add(search_textbox);
this.Controls.Add(search_icon_image);
}

1 reference
public void ClearSearch(object sender, EventArgs e)
{
    for (int i = 0; i < this.Controls.Count; i++) if (this.Controls[i].Name == "SearchTextbox") this.Controls[i].Text = "";
}
```

We next needed to determine which cars match the criteria given in the search bar. I went ahead and did a contains on each of the manufacturer, model and description properties. Please note how you get around case issues with the ToUpper method. Oddly enough, I was not able to reference the TextBox directly via name, still not sure why that happened but I was able to easily get around it as there was only one TextBox in our Controls collections. (Figure 15A.10)

Figure 15A.10: WhichCars

```
1 reference
private void ExecuteSearch()
{
    WhichCars();
    DisplayMatches();
}

1 reference
private void WhichCars()
{
    Array.Clear(displayed_cars, 0, displayed_cars.Length);

    for (int i = 0; i < displayed_cars.Length; i++) displayed_cars[i] = -1; // indicates no car

    // for some reason Visual Studio does not recognize the SearchTextbox name so I will force it
    foreach (Control c in this.Controls)
    {
        if (c is TextBox)
        {
            TextBox txt = (TextBox)c;
            int counter = 0;
            foreach (car a_car in my_cars)
            {
                if (a_car.Manufacturer.ToUpper().Contains(txt.Text.ToUpper()) ||
                    a_car.Model.ToUpper().Contains(txt.Text.ToUpper()) ||
                    a_car.description.ToUpper().Contains(txt.Text.ToUpper()))
                {
                    if (counter < 8) // we only have room for 9 cars
                    {
                        displayed_cars[counter] = a_car.ID;
                        counter++;
                    }
                }
            }
        }
    }
}
```

We complete our searchbar with the DisplayMatches routine. (Figure 15A.11) Here we are back to those nested foreach control loops. Again, I can make it work but I am starting to believe Microsoft when they say nesting TableLayoutPanels isn't such a great idea. I certainly wouldn't want to turn this code over to a junior programmer for maintenance. The nesting is just too obtuse.

Figure 15A.11: DisplayMatches

```
private void DisplayMatches()
{
    car this_car = new car();
    if (displayed_cars[0] == -1) // no matches
        MessageBox.Show("There were no matches to the entered criteria.");
    else
    {
        for (int i = 0; i < displayed_cars.Length; i++)
        {
            this_car = null;
            if (displayed_cars[i] >= 0) // is a valid car ID
            {
                foreach (car a_car in my_cars)
                {
                    if (a_car.ID == displayed_cars[i]) this_car = a_car; // this_car is now pointing to a car to display
                }
            }

            if (this_car != null)
            {
                bool applied;
                foreach (Control a in this.Controls)
                {
                    applied = false;
                    if (a is TableLayoutPanel) // found parent
                    {
                        foreach (Control b in a.Controls)
                        {
                            if (b is TableLayoutPanel) // found child
                            {
                                PictureBox p1 = (PictureBox)b.Controls[0]; // we know this because we know order we created objects in child grid
                                if (p1.Image == null && applied == false)
                                {
                                    p1.Image = Image.FromFile (this_car.pics[0].filename);
                                    Label l1 = (Label)b.Controls[1];
                                    l1.Text = this_car.Manufacturer + " " + this_car.Model;
                                    Label l2 = (Label)b.Controls[2];
                                    l2.Text = this_car.price.ToString("C", new CultureInfo("en-US"));
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}
```

We wrap up our work on this front page by putting together that ImageWasClicked event handler. (Figure 15A.12) Back to nice easy code here.

Figure 15A.12: ImageWasClicked

```
9 references
public void ImageWasClicked (object sender, EventArgs e, int index)
{
    this.Hide();
    CarDetails newPage = new CarDetails(my_cars[index]);
    newPage.ShowDialog();
    this.Close();
}
```

This concludes our work on our home page. In our next chapter we will take a look at the CarDetails page.